

# gemstone-py Framework Adapters

Adding sync and async web framework adapters

Generated from the Markdown sources in gemstone-py/docs  
Assets are local SVG illustrations stored in the repository.

# Contents

Adding a Framework Adapter

Sync Framework Shape

Async Framework Shape

Existing Adapters

# Adding a Framework Adapter

---

Use `gemstone_py.web_core` for request lifetime and transaction decisions. Adapters should stay thin: translate the framework's request hooks into `RequestScope` or `AsyncRequestScope`, then expose one helper that returns the current request session.

Keep retry policy in the application service layer. The adapter should own session acquisition and final commit/abort behaviour; it should not silently replay request handlers.

## Sync Framework Shape

```
from gemstone_py import GemStoneConfig, RequestScope, TransactionPolicy
from gemstone_py.session_providers import GemStoneSessionPool

pool = GemStoneSessionPool(maxsize=4, config=GemStoneConfig.from_env())

def begin_request(request):
    request.gemstone_scope = RequestScope(
        session_provider=pool,
        transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
    )

def current_session(request):
    return request.gemstone_scope.session()

def end_request(request, exc=None, response_status=None):
    request.gemstone_scope.finalize(exc, response_status=response_status)
```

The scope commits on successful responses, aborts on exceptions or server-error status codes, and releases pooled sessions with the correct `clean` / `discard` flags.

When a specific write workflow needs bounded conflict replay, wrap that workflow with `retrying_transaction(...)` outside the request scope or in a service function that can safely rerun the whole unit of work:

```
from gemstone_py import retrying_transaction

def save_booking(session):
    ...

retrying_transaction(save_booking, config=GemStoneConfig.from_env(), attempts=5)
```

Do not retry an already-running request scope unless the whole request body can be safely replayed after aborting and reloading state.

# Async Framework Shape

```
from gemstone_py import AsyncRequestScope, GemStoneConfig, TransactionPolicy
from gemstone_py.aio.pool import AsyncSessionPool

pool = AsyncSessionPool(maxsize=8, config=GemStoneConfig.from_env())

async def begin_request(scope):
    scope["gemstone_scope"] = AsyncRequestScope(
        session_provider=pool,
        transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
    )
    scope["gemstone_session"] = await scope["gemstone_scope"].session()

async def end_request(scope, exc=None, response_status=None):
    await scope["gemstone_scope"].finalize(exc, response_status=response_status)
```

## Existing Adapters

| Framework | Module                                     | Pattern                              |
|-----------|--------------------------------------------|--------------------------------------|
| Flask     | <code>gemstone_py.frameworks.flask</code>  | request-local sync session           |
| Django    | <code>gemstone_py.frameworks.django</code> | middleware-managed sync session      |
| FastAPI   | <code>gemstone_py.aio.fastapi</code>       | async dependency and ASGI middleware |
| Litestar  | <code>gemstone_py.aio.litestar</code>      | async dependency and ASGI middleware |

Keep optional framework imports inside app/example code or function bodies, not at package import time. The adapter module should remain importable in a plain `gemstone-py` installation.

One GemStone session belongs to one active execution path. Framework adapters should make session sharing explicit through a pool or request-local provider, not by storing a session in a global variable.

This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.